UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE AND TECHNOLOGY

SCHOOL OF ICT

DEPARTMENT: COMPUTER ENGINEERING

MODULE: MOBILE APPLICATION SYSTEMS AND DESIGN

LAB 5: OFFLINE POST MANAGER

DATE 20th MARCH 2026

REGISTRATION NUMBERS:

Ronsard RUGIRANGOGA 223026673

1. Handwritten Discussions

Reg number 22 3026673

Q1. Dependency used & why SQllite is Necessary?

Dependency	purpose
sqflite	SQlite database for local storage
path_provider	Finds device storage location
path	Joins file paths correctly
fluttertoast	Shows success/error notifications
cupertino_icons	professional UI icons

Why SQLite is Necessary for local storage?:

Offline access where as staff work anywhere without internet
Data persistence - posts remain after app closes/restarts
Multi-User support ; each user has private posts with foreign keys
Privacy where by data stays on device not in cloud

Q2) Handling database exceptions

Database not initialized
- Splash screen initializes database before showing UI
- check database instance before ~~everywhere~~ operation
- Retry option with error messages when fails

Inserts / Update / Delete errors:
- Try - catch blocks around all database operations
- Data validation before inserting (check empty fields)
- toast notifications (with green or red ~~colour~~)

Invalid or Corrupted data:
- Type validation using int.tryParse(), DateTime.TryParse
- Default values for null/missing fields

Q3: Explain The future widget you have used, And part of the UI depends on the Future submission requirements

A future represents a value that will be Available later.
Used for operations that take time like API calls.
The App continue running while Waiting.

How Future were used:

Method used: Manual state management with setstate

~~3 variable~~

3 state Variables

1. posts - Stores the data when future completes
2. _is loading - true while waiting, false when done
3. _error Message - Stores error if future fails

UI parts depends on the future

loading Indicator : when waiting for API Response
Post list display : list of postcards with title and content ~~when~~ After future completes
Error Message Display : Show error icons, error message when future completes with an error
Create post button state : UI : Button shows loading spinner, when : while waiting for create operation to complete
updating post Button State : ,, ,, ,, ,,
Delete operation : UI (confirmation dialog, then loading Indicator)
when : while waiting for delete confirmatio
Refresh operation : - Refresh operator at top of list (UI)
- while fetching Fresh data from ~~API~~ (while

2. App Screenshots

All 11 screenshots below demonstrate the complete functionality of the Offline Posts Manager application.

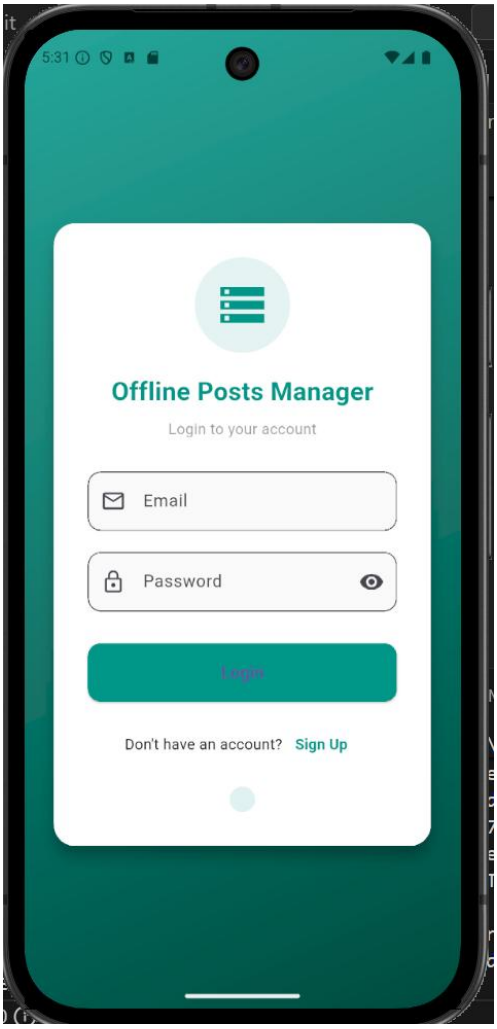


Fig 1. Login screen

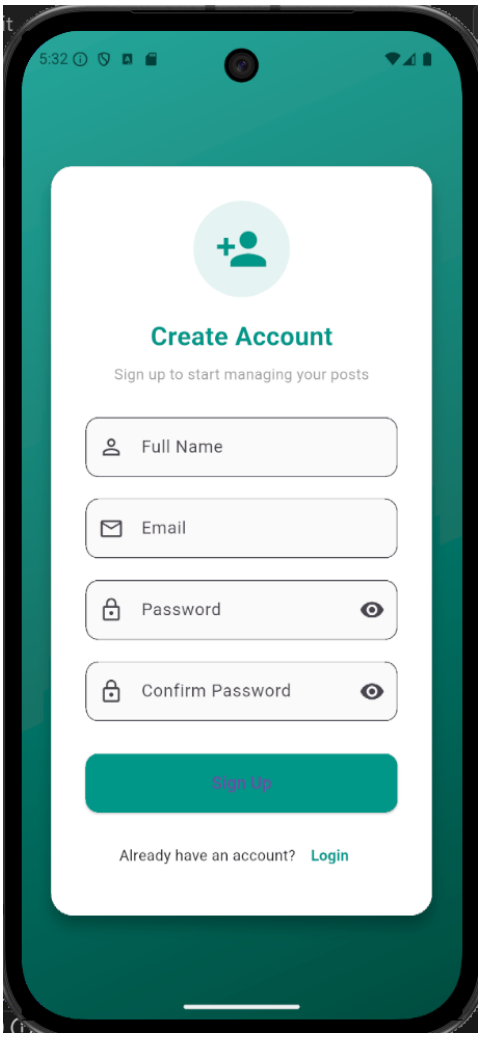


Fig 2. Register screen

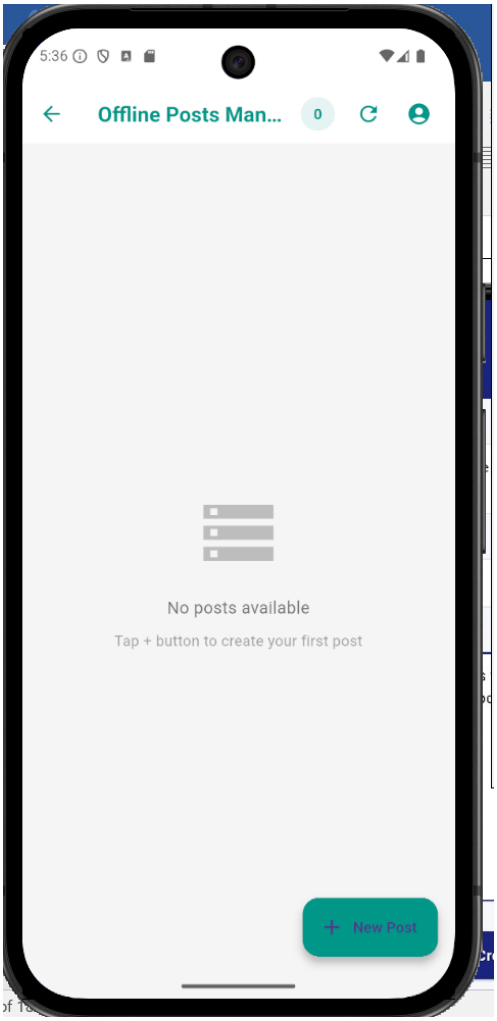


Fig 3. Posts dashboard

Fig 4. Add post

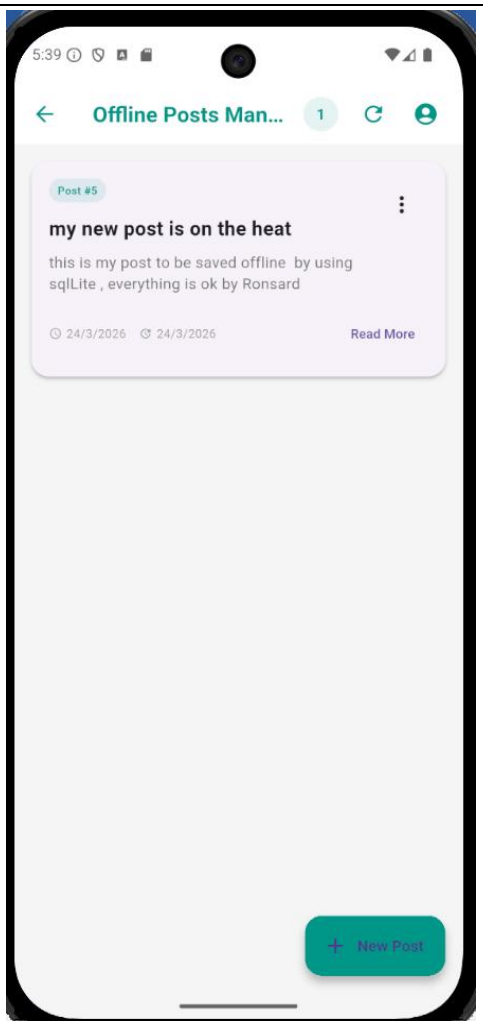


Fig 5. Added posts

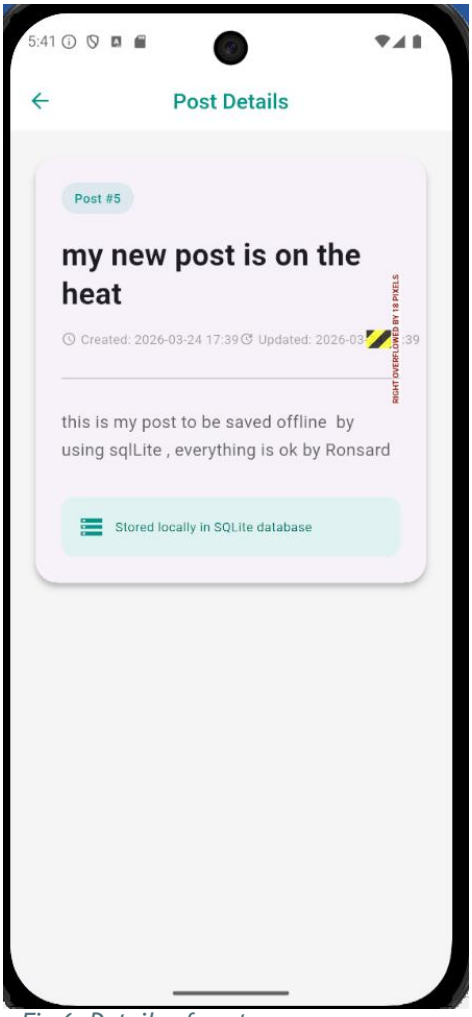


Fig 6. Details of post

Fig 7. Edit existing post

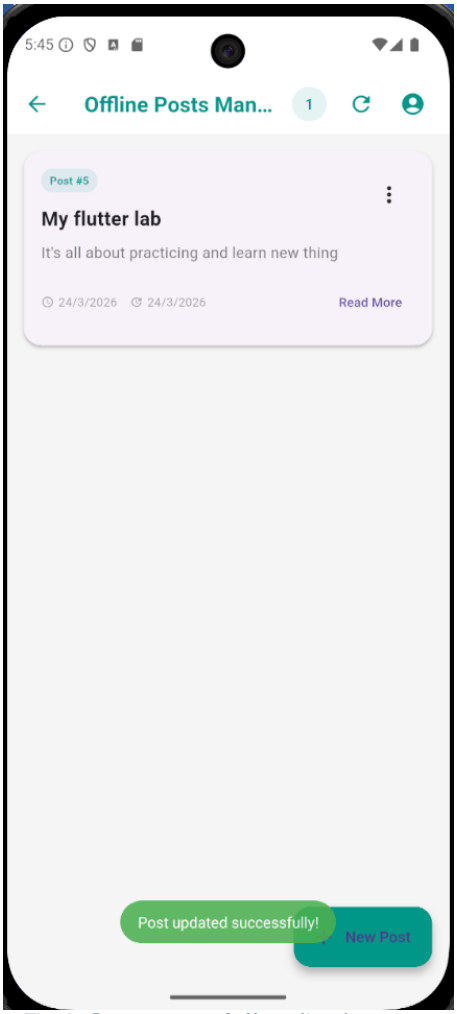


Fig 8. Post successfully edited

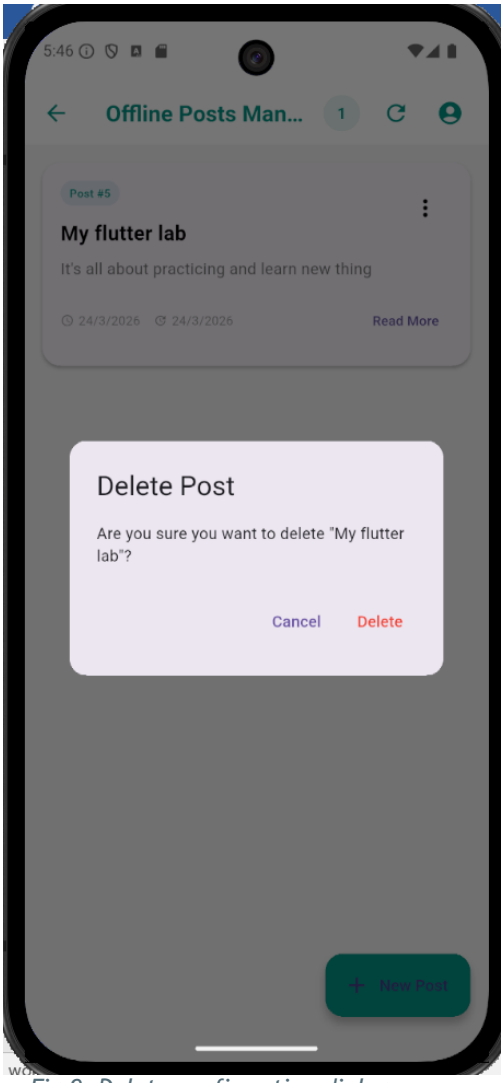


Fig 9. Delete confirmation dialog

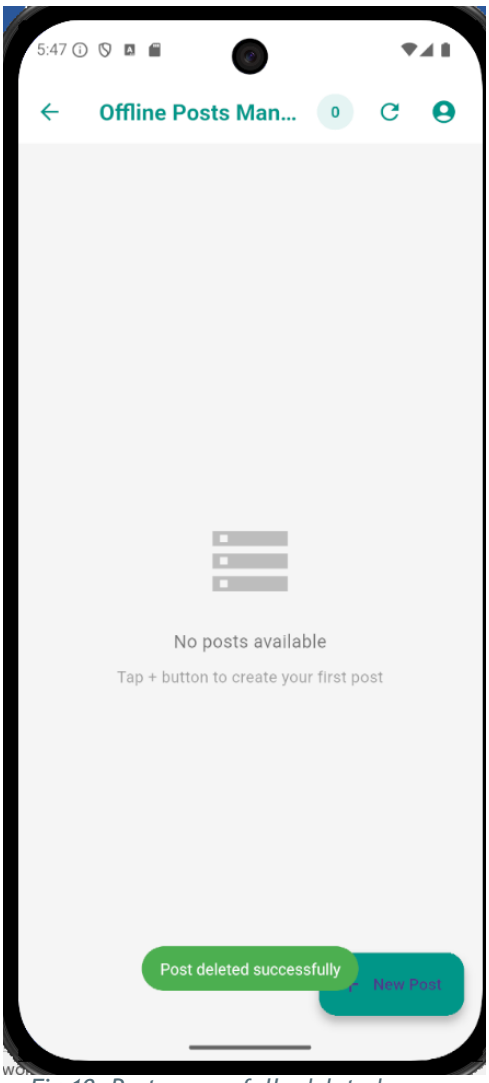
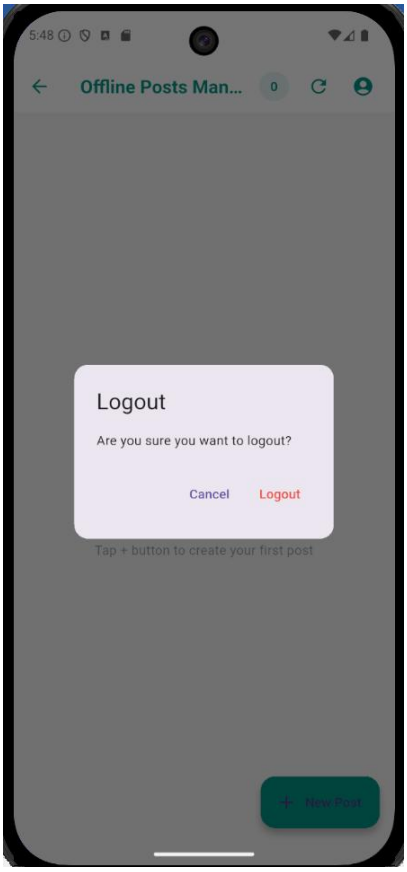


Fig 10. Post successfully deleted



6. Conclusion

This lab successfully demonstrates SQLite as a local storage solution in Flutter. The Offline Posts Manager implements all five required CRUD features plus user authentication (register, login, logout) entirely offline. The implementation covers robust exception handling at three levels, asynchronous database interaction via async/await and Futures, and a clean layered architecture separating models, database, services, and UI screens.

References

- SQLite Official Documentation — https://www.sqlite.org/docs.html
- Dart Async/Await — https://dart.dev/codelabs/async-await
- sqflite package — https://pub.dev/packages/sqflite • path package — https://pub.dev/packages/path
- crypto package — https://pub.dev/packages/crypto